private links with others. Often such risks are not clear to the end user, and more often than not they shouldn't be there in the first place. For example, a user browsing a shopping site might assign a cart id under which all your items are added. The user then shares a link to his cart page with a friend who might be interested the same. Since this link uses includes the cart id, the friend is unknowingly using the same cart, and when he purchases from that cart, the money is deducted from the original users account.

This is a huge flaw in the system, and might be more common in web sites that allow a user to shop without needing to register.

A malicious user could use such links that include the session id or cart id to hijack the identity of the original user and conduct transactions on their behalf. If such session ids are not random enough, an attacker might be able to guess the session ids of others, or even brute-force their way to a valid session id and then take on the identity of such a user. Sessions should be timed out appropriately, so that a user who has simply closed the browser window without logging out first is protected.

While both the parameters sent via GET in the URL and those POSTed can be inspected by an attacker, GET parameters occur in the URL and will be stored in the history in the browser, and in the cache. This information can be invaluable to an attacker.

Error messages that a web site returns can also be a source of attack. While it may seem helpful for you to give the information as much information as possible, this information can also be used by an attacker. Any sensitive information that establishes the identity of a user should be transmitted over an encrypted connection.

For example, when you enter an incorrect password, the application specifies that while the login id is correct, the password is not. What is wrong with this picture? Well, for one you have just exposed that the user has an account on this website! This is why most websites will serve a message like "Invalid user ID or password." It is not enough that the message that the user can see does not expose anything, but nothing in the URL, or the source of the document or the request structure should betray this information either. If an attacker can establish that the website takes more time to process an invalid password than it takes to process an invalid username, then that is information enough for the attacker.

For an attacker who already has a login, the process of breaking the password should not be simple. If a website allows a user indefinite tries for logging in, then it is vulnerable to a password cracking attack. It is not only important that the login page be secure from such attacks, but also the "forgot password" page. If an attacker has indefinite tries to guess a user's password, then eventually he will simply be able to guess the right values.

Also important is that you store all passwords as hashed values – salted